

Technical Report: ToS Semantic Similarity & Unfair Clause Detection

1. Problem Statement

Terms of Service documents are long, opaque, and rarely read by consumers. Yet they frequently contain clauses that are potentially unfair under EU consumer law — granting providers the right to unilaterally terminate accounts, modify terms without notice, limit liability, or impose foreign jurisdiction on disputes. This project builds a system to automatically detect such clauses and enable semantic search over a corpus of 50 real Terms of Service documents.

One hard constraint applies throughout: no generative language models and no retrieval-augmented generation. The system relies entirely on classical machine learning and embedding-based retrieval.

2. Dataset

The corpus is drawn from the CLAUDETTE project (Lippi et al., 2019) — 50 Terms of Service documents from major online platforms including Google, Amazon, Netflix, Spotify, and Uber, segmented into sentences using Stanford CoreNLP and annotated by legal experts. Sentences shorter than five tokens are discarded, yielding a working corpus of **9,414 sentences**, of which **1,032 (11.0%)** are labeled as potentially unfair.

Labels exist at two levels of granularity:

- **General label:** binary unfair / fair per sentence
- **Category labels:** eight separate binary labels, one per unfairness category:

Symbol	Category
A	Arbitration
CH	Unilateral Change
CR	Content Removal
J	Jurisdiction
LAW	Choice of Law
LTD	Limitation of Liability
TER	Unilateral Termination
USE	Contract by Using

The class imbalance is significant at roughly 9:1 (fair to unfair). All models account for this explicitly through cost-sensitive weighting rather than data augmentation — see Section 3.3.

3. Classification

3.1 Feature Representation

All classifiers use **TF-IDF with unigrams and bigrams** as the feature representation (`ngram_range=(1,2)`, `sublinear_tf=True`, `min_df=2`, `max_features=50,000`). This is consistent with the CLAUDETTE paper, which found that lexical bag-of-words features outperform both syntactic tree kernels and neural embeddings for this domain. The logic is: unfair clauses share a distinctive vocabulary, phrases like "*at our sole discretion*", "*we reserve the right*", and "*in no event shall*", that is highly discriminative and well-captured by n-gram statistics.

3.2 Classification Strategies

Two strategies were implemented, corresponding to classifiers C1 and C2 in the CLAUDETTE paper:

- **C1) Single classifier:** A single binary classifier trained on the general unfairness label. One model per fold, predicting fair or unfair for each sentence.
- **C2) Combined classifiers (OR rule):** Eight binary classifiers, one per unfairness category, each trained on its corresponding category label. A sentence is predicted as unfair if *any* of the eight classifiers flags that sentence as unfair. This approach allows each classifier to specialise on the lexical patterns of one category, and the OR combination recovers the general unfairness prediction.

3.3 Handling Class Imbalance

No oversampling (e.g. SMOTE) was applied. Instead, cost-sensitive weighting was used throughout:

- **SVM, Logistic Regression, Random Forest:** `class_weight='balanced'`, which automatically scales the loss penalty by $n_samples / (n_classes \times n_samples_per_class)$. At a 9:1 ratio, the minority class receives approximately 9x the weight of the majority class during training.
- **XGBoost:** `scale_pos_weight = n_negatives / n_positives`, computed from the actual training fold distribution. This is XGBoost's equivalent mechanism.

SMOTE was not used because interpolating between sparse TF-IDF vectors does not produce semantically meaningful synthetic sentences, and cost-sensitive weighting is simpler and equally effective in high-dimensional sparse spaces.

3.4 Evaluation Protocol

5-fold document-level cross-validation was used: the 50 documents are split into 5 folds of 10 documents each. The TF-IDF vectorizer and all classifiers are fit once per fold on the 40 training documents and evaluated on the 10 test documents. Documents with zero positive labels in the test fold are excluded from metric computation, as precision and recall are undefined for them.

The reported metric is **macro-averaged Precision, Recall, and F1** across all evaluated test documents.

Note: The original CLAUDETTE paper used leave-one-document-out (LOO) cross-validation with 50 folds. The 5-fold approach used here is a practical approximation that produces directionally consistent results with significantly less computation.

3.5 Results

C1 — Single Classifier

Model	Precision	Recall	F1
SVM	0.737	0.772	0.749
Logistic Regression	0.650	0.832	0.724
Random Forest	0.924	0.433	0.577
XGBoost	0.582	0.765	0.653

C2 — 8 Category Classifiers (OR-combined)

Model	Precision	Recall	F1
SVM	0.761	0.757	0.754
Logistic Regression	0.613	0.868	0.712
Random Forest	0.925	0.253	0.381
XGBoost	0.573	0.782	0.653

3.6 Analysis

SVM (C2) achieves the best overall F1 at 0.754 and is selected as the production classifier. The category-decomposition strategy of C2 provides a small but consistent improvement over C1-SVM (0.754 vs 0.749 in F1), while also producing per-category predictions as a useful side effect.

Logistic Regression achieves the highest recall (0.868 in C2) but at the cost of substantially lower precision (0.613). It classifies a large number of fair sentences as unfair, making it less useful in practice unless recall is the dominant concern.

Random Forest has the best precision (0.925 in C2) of all models, but also the worst recall (0.253). Despite `class_weight='balanced'`, the ensemble of decision trees produces an overly conservative model that classifies nearly everything as fair. This is a known limitation of tree-based ensembles in very high-dimensional sparse feature spaces — individual trees cannot effectively split on sparse TF-IDF features the way linear models can.

XGBoost consistently underperforms both SVM and Logistic Regression across both strategies. Like Random Forest, gradient boosting over decision trees struggles with the sparsity of TF-IDF representations.

The results confirm the CLAUDETTE paper's finding: **linear classifiers over bag-of-words features are the most effective approach** for this task. The discriminative vocabulary of unfair clauses makes lexical features sufficient, and linear models generalise well from them.

4. Semantic Similarity Search

4.1 Approach

To support retrieval of similar corpus sentences for a given query, two embedding strategies were compared:

- **TF-IDF cosine similarity:** The same TF-IDF vectorizer used in classification, fit on the full corpus. Vectors are L2-normalised; cosine similarity reduces to a dot product.
- **Sentence-BERT (SBERT):** The `all-MiniLM-L6-v2` model from `sentence-transformers`, producing dense 384-dimensional embeddings. Vectors are L2-normalised by the model.

Retrieval uses brute-force cosine similarity over the full corpus matrix (~9,400 sentences). At this scale, sparse matrix cosine similarity completes in milliseconds, so no approximate nearest-neighbour index is needed.

4.2 Evaluation Protocol

The 50 documents are split 80/20 at the document level (40 train, 10 test, random seed 42). The embedding index is built on training sentences only. Each unfair sentence in the test set is used as a query (188 total queries). The following metrics are computed:

- **P@5:** Fraction of the top-5 retrieved sentences that are labeled unfair
- **P@10:** Fraction of the top-10 retrieved sentences that are labeled unfair
- **MAP:** Mean Average Precision, measuring the quality of the full ranking

4.3 Results

Method	Queries	P@5	P@10	MAP
TF-IDF	188	0.606	0.571	0.695
SBERT	188	0.585	0.536	0.675

4.4 Analysis

TF-IDF outperforms SBERT on all three metrics. This result is counter-intuitive at first glance. SBERT is a powerful semantic model trained on hundreds of millions of sentence pairs, but it is entirely consistent with the classification findings.

The explanation is domain-specific: identifying unfair clauses is fundamentally a lexical task. A known unfair clause retrieved by a query tends to share the exact same keywords and phrases ("*terminate*", "*modify*", "*discretion*", "*reserve the right*"). SBERT retrieves sentences that are semantically paraphrased, capturing the general *meaning*, but in a domain where the specific legal phrasing carries the signal, semantic generalisation actually hurts precision.

SBERT's strengths (cross-lingual transfer, paraphrase robustness, understanding of sentence structure) provide no advantage in a monolingual, keyword-driven legal corpus. TF-IDF's strength, which is sensitivity to exact term frequency, is precisely what the task rewards.

TF-IDF was selected as the default embedding method for the service. SBERT remains available as an optional alternative via the `EMBEDDING_METHOD=sbert` environment variable for use cases where paraphrase robustness is more important than retrieval precision.

5. Service Architecture

5.1 Overview

The system is packaged as a containerised REST API with a web frontend:

```
docker-compose
├─ api (FastAPI, port 8000) ← corpus index + C2-SVM classifier
└─ app (Streamlit, port 8501) ← web UI, calls /analyze
```

The API exposes four endpoints:

Endpoint	Description
GET /health	Service status, active embedding method, corpus size
POST /search	Top-k similar sentences for a query string
POST /classify	C2-SVM prediction with per-category breakdown
POST /analyze	Combined: classification result + similar sentences

On startup, the service loads the full corpus, builds the embedding index, and trains or loads from a disk cache the eight SVM classifiers. First startup takes approximately 10–20 seconds. Subsequent restarts load cached models from `data/models/` in under a second. The Streamlit container waits for the API healthcheck to pass before becoming available.

5.2 Key Design Decisions

Decision	Choice	Rationale
Similarity backend	Sparse brute-force cosine	Sufficient for ~9,400 sentences; avoids FAISS as an additional dependency
Default embedding	TF-IDF	Higher retrieval quality per experiment (MAP 0.695 vs 0.675); zero model download; instant startup
Production classifier	C2-SVM	Best F1 (0.754); produces per-category predictions without additional cost
Model persistence	joblib	Native scikit-learn serialisation; trivially handles sparse vectorizers and LinearSVC
API framework	FastAPI	Async lifespan for startup logic, automatic OpenAPI documentation, Pydantic request validation
Frontend	Streamlit	Minimal code; no JavaScript required; appropriate for research prototypes
Containerisation	docker-compose (2 services)	Clean separation between API and UI; health-check dependency ensures correct startup order

6. Limitations

Evaluation diverges from the original paper. The 5-fold cross-validation used here is a practical approximation of the 50-fold leave-one-document-out procedure in CLAUDETTE. Results are directionally consistent but not directly comparable due to differences in fold count and variance.

Retrieval does not generalise across phrasings. Because TF-IDF is purely lexical, a user who queries with different phrasing than what appears in the corpus will receive lower-quality results. SBERT partially mitigates this at the cost of lower precision, and is available as a swap-in via environment variable.

Corpus size is limited. The 50-document corpus is representative but small. The service is most useful for finding precedents within this curated set, not for open-domain legal reasoning.

7. How to Run

```
# Install dependencies
pip install -r requirements.txt

# Run classification experiments
python main.py

# Start the API
uvicorn service.api:app --reload
# → Interactive docs at http://localhost:8000/docs

# Start the Streamlit UI (separate terminal)
streamlit run app.py
# → http://localhost:8501

# Run the full stack with Docker
docker-compose up
# API → localhost:8000 | UI → localhost:8501

# Run the embedding comparison experiment
python scripts/embedding_experiment.py
```

In case of problems running the code, the project was also deployed on streamlit cloud community and the API was deployed using Render. Since it has a cold-start, the first time using the app on streamlit, the API may take a few seconds to be up and running. It is recommended to make a request to activate it and wait a few seconds.

The webapp can be accessed via the following link: <https://tos-sentence-classification.streamlit.app/>

References

Lippi, M., Palka, P., Contissa, G., Lagioia, F., Micklitz, H.-W., Sartor, G., & Torroni, P. (2019). *CLAUDETTE: An Automated Detector of Potentially Unfair Clauses in Online Terms of Service*. Artificial Intelligence and Law. <https://doi.org/10.1007/s10506-019-09243-2>